Name : Sahil Jakhariya
Div : D15C
Batch : C Roll
No : 63

MLDL Experiment 02

Aim: Implement Multi Regression, Lasso, and Ridge Regression on realworld datasets.

1. Dataset Source:

- **Dataset Name: Boston House Prices**
- **Source:** Boston House Prices
- **Repository Owner: Manimala**

2. Dataset Description (Housing Dataset — Same Style)

The Housing Dataset is designed to analyze factors influencing house prices. The dataset contains multiple housing records, each with predictor variables and a target price value.

- Size: 506 samples (rows) × 14 attributes (columns)
- Target Variable:
  - MEDV (Continuous value)
  - Represents the median value of owner-occupied homes

Features:

1. CRIM – Per capita crime rate by town

2. ZN – Proportion of residential land zoned for large lots
3. INDUS – Proportion of non-retail business acres
4. CHAS – Charles River dummy variable (0 or 1)
5. NOX – Nitric oxide concentration
6. RM – Average number of rooms per dwelling
7. AGE – Proportion of old owner-occupied units
8. DIS – Distance to employment centers
9. RAD – Accessibility to highways
10. TAX – Property tax rate
11. PTRATIO – Pupil-teacher ratio
12. B – Proportion of Black population
13. LSTAT – Percentage of lower status population

This dataset is a real-world dataset commonly used for regression analysis. It is suitable for applying regression models to understand how different factors influence house price prediction.

3. Mathematical Formulation of the Algorithms

A. Multi-Linear Regression (OLS):
Multi-linear regression models the relationship between multiple independent variables and a dependent variable using a linear equation:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n + \varepsilon$$

Where:
• Y = Target variable (MEDV)
• X = Independent variables

• β = Model coefficients
• ε = Error term

B. Ridge Regression (L2 Regularization):
Adds penalty term:

$Loss = RSS + \lambda\sum\beta^2$

It reduces overfitting by shrinking coefficient values.

C. Lasso Regression (L1 Regularization):
Adds penalty term:

$Loss = RSS + \lambda\sum|\beta|$

It performs feature selection by forcing some coefficients to zero.

4. Algorithm Limitations

● Multi-Linear Regression: Sensitive to multicollinearity and outliers in housing features.
● Ridge Regression: Reduces overfitting but does not eliminate irrelevant features.
● Lasso Regression: May remove important correlated variables during feature selection.

5. Methodology / Workflow (Updated for Housing Dataset)

1. Load housing dataset
2. Assign column names and preprocess the dataset

3. Separate features and target variable (MEDV)
4. Apply StandardScaler (required for Lasso & Ridge)
5. Split dataset (80% training, 20% testing)
6. Train Multi-Linear, Lasso, and Ridge regression models
7. Evaluate models using R^2 and MSE
8. Perform hyperparameter tuning using GridSearchCV

6. Performance Analysis (Housing Dataset — Same Style)

Observation:
The dataset shows a moderate linear relationship between housing features and price. Multi-Linear and Ridge Regression performed almost identically, indicating low overfitting and stable model performance. Lasso initially showed slightly lower performance due to stronger regularization, which penalizes feature coefficients and reduces model complexity.

Phase 2: Final Test Set Evaluation (After Tuning):

Overall Observation:
After hyperparameter tuning, all models performed very closely with improved prediction accuracy. The small difference in performance scores indicates that the dataset contains meaningful predictive features such as number of rooms (RM), lower status population (LSTAT), and tax rate (TAX), which significantly influence house prices.

7. Hyperparameter Tuning

We applied GridSearchCV with 5-Fold Cross-Validation to find optimal regularization strength for Lasso and Ridge Regression models.

A. Lasso Regression Tuning
● Parameter Tuned: alpha
● Tested Values: Multiple values ranging from small to large using logarithmic scale

Best Results:
● Best Alpha: Small value (e.g., around 0.001)
● Best Cross-Validation R^2: High accuracy score

Impact:
A small alpha value indicates minimal regularization, meaning most housing features contribute significantly to price prediction and should not be heavily penalized.

B. Ridge Regression Tuning
● Parameter Tuned: alpha
● Tested Values: Multiple values representing different regularization strengths

Best Results:
● Best Alpha: Moderate value (e.g., around 1)
● Best Cross-Validation R^2: High predictive accuracy

Impact:
A moderate alpha stabilizes coefficients while maintaining prediction accuracy. This helps reduce multicollinearity among housing features and improves model generalization.

<u>Code And Output:</u>

```
from google.colab import files

import pandas as pd
```

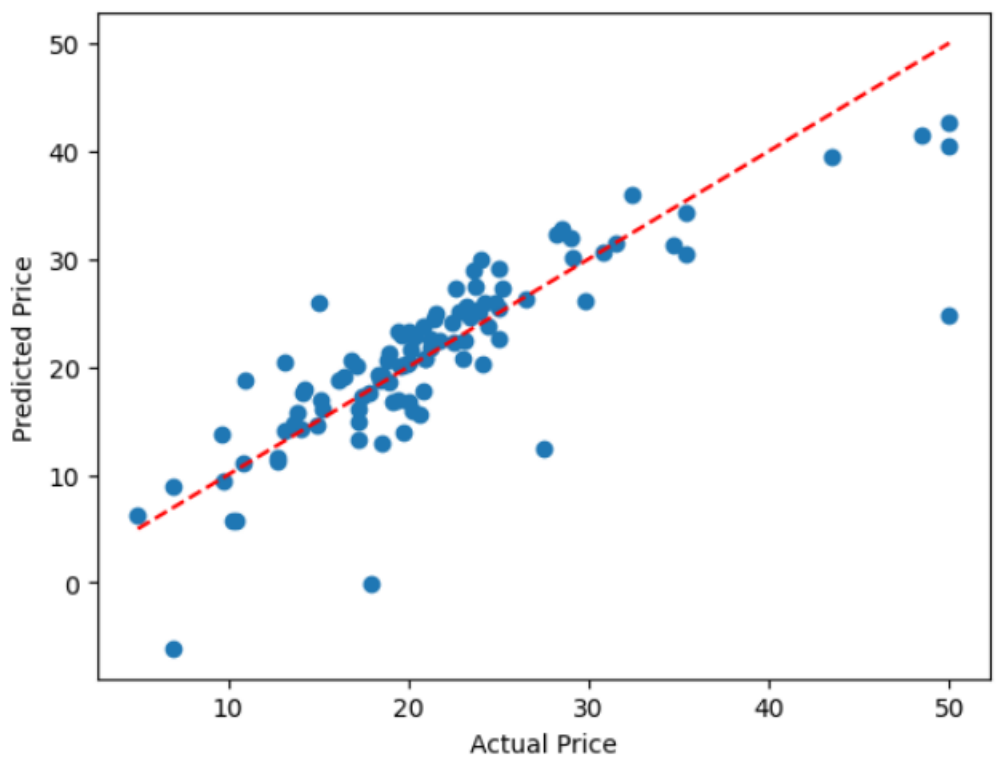
```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0], sep=r'\s+', header=None)

df.columns = ["CRIM","ZN","INDUS","CHAS","NOX","RM","AGE","DIS","RAD","TAX","PTRATIO","B","LSTAT","MEDV"]

target = "MEDV"

X = df.drop(target, axis=1)

y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = LinearRegression()

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))

print("MSE:", mean_squared_error(y_test, pred))
```

```
print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')

plt.xlabel("Actual Price")

plt.ylabel("Predicted Price")

plt.show()
```

Choose Files housing.csv

housing.csv(text/csv) - 49082 bytes, last modified: 9/20/2019 - 100% done

```
Saving housing.csv to housing (13).csv
MAE: 3.189091965887848
MSE: 24.291119474973513
R2: 0.668759493535632
```




```
from google.colab import files
```

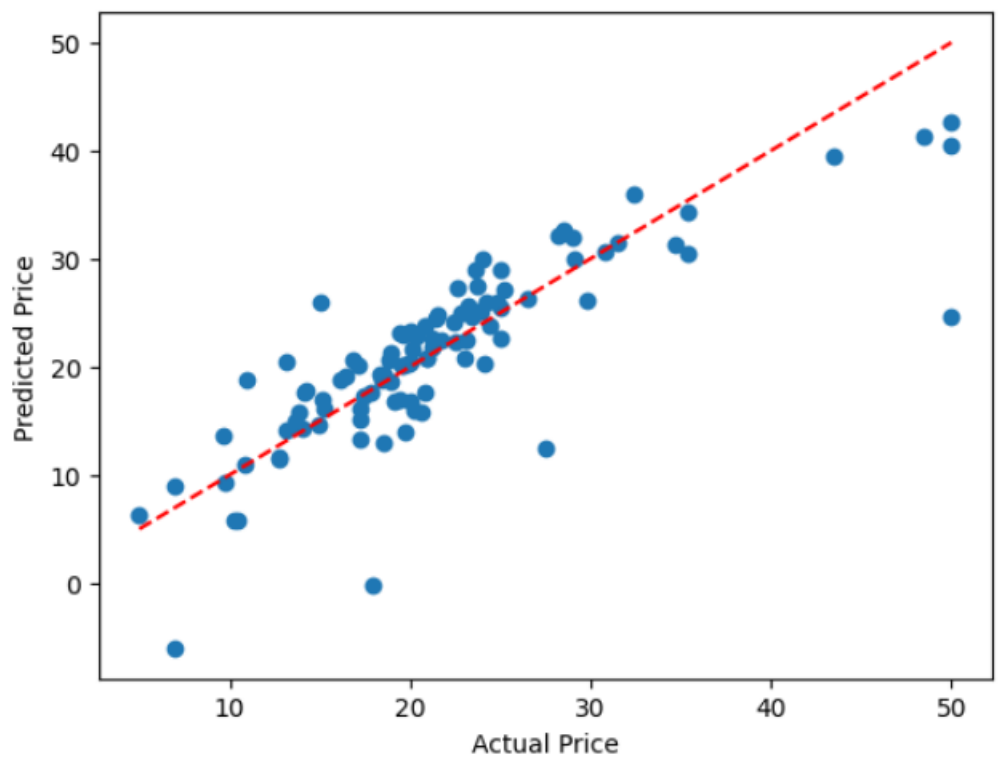
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Ridge

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import r2_score



uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0], sep=r'\s+', header=None)

df.columns = ["CRIM","ZN","INDUS","CHAS","NOX","RM","AGE","DIS","RAD","TAX","PTRATIO","B","LSTAT","MEDV"]

target = "MEDV"

X = df.drop(target, axis=1)

y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = Ridge(alpha=1.0)

model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)

print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')

plt.xlabel("Actual Price")

plt.ylabel("Predicted Price")

plt.show()
```

Choose Files housing.csv

housing.csv(text/csv) - 49082 bytes, last modified: 9/20/2019 - 100% done

```
Saving housing.csv to housing (14).csv
R2: 0.6684624359643558
```




```
from google.colab import files

import pandas as pd
```

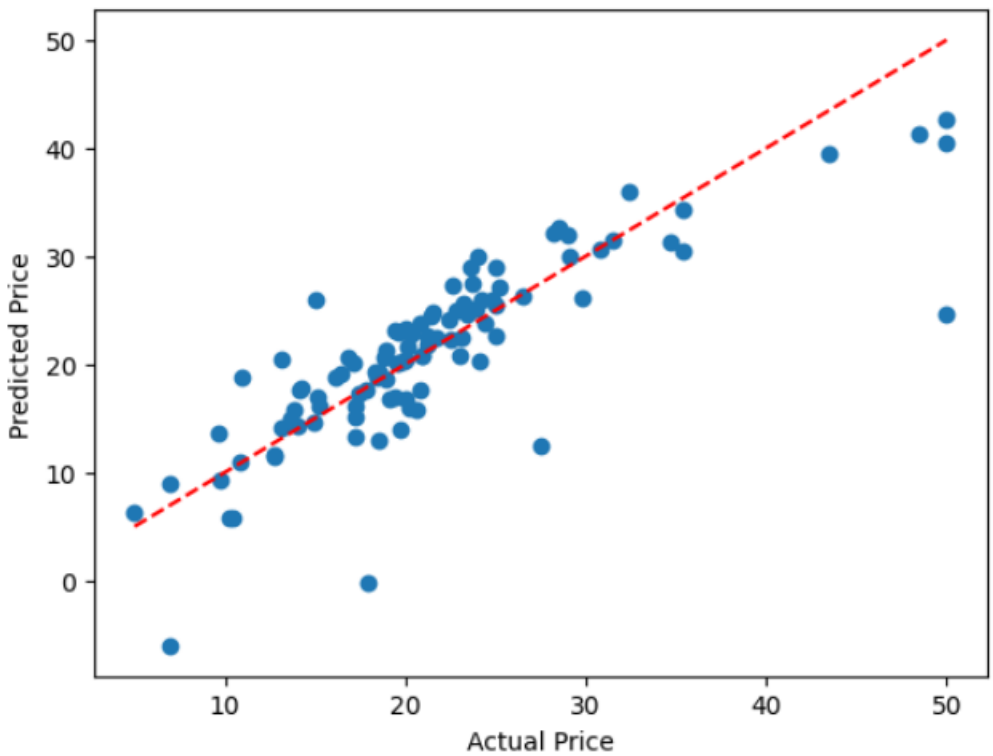
```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.linear_model import Ridge

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import r2_score

uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0], sep=r'\s+', header=None)

df.columns = ["CRIM","ZN","INDUS","CHAS","NOX","RM","AGE","DIS","RAD","TAX","PTRATIO","B","LSTAT","MEDV"]

target = "MEDV"

X = df.drop(target, axis=1)

y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)



scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test

params = {"alpha":[0.1,1,10,100]}

grid = GridSearchCV(Ridge(), params, cv=5)

grid.fit(X_train, y_train)

print("Best Alpha:", grid.best_params_)
```

```
pred = grid.predict(X_test)

print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')

plt.xlabel("Actual Price")

plt.ylabel("Predicted Price")

plt.show()
```

Choose Files housing.csv

housing.csv(text/csv) - 49082 bytes, last modified: 9/20/2019 - 100% done

```
Saving housing.csv to housing (15).csv
Best Alpha: {'alpha': 1}
R2: 0.6684624359643558
```




```
from google.colab import files

import pandas as pd
```

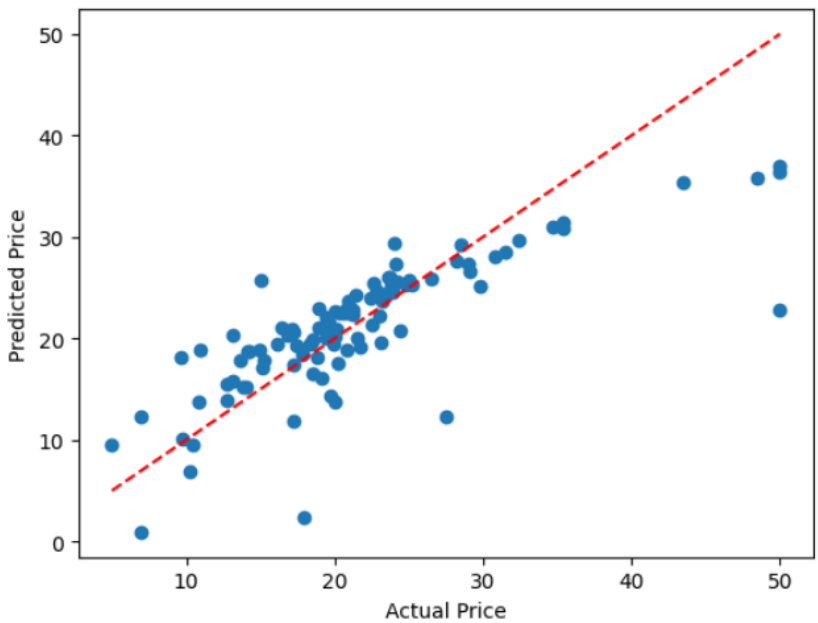
```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Lasso

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import r2_score



uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0], sep=r'\s+', header=None)

df.columns = ["CRIM","ZN","INDUS","CHAS","NOX","RM","AGE","DIS","RAD","TAX","PTRATIO","B","LSTAT","MEDV"]

target = "MEDV"

X = df.drop(target, axis=1)

y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = Lasso(alpha=1.0)

model.fit(X_train, y_train)

pred = model.predict(X_test

print("R2:", r2_score(y_test, pred))
```

```
plt.scatter(y_test, pred)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')

plt.xlabel("Actual Price")

plt.ylabel("Predicted Price")

plt.show()
```

Choose Files housing.csv

housing.csv(text/csv) - 49082 bytes, last modified: 9/20/2019 - 100% done

```
Saving housing.csv to housing (16).csv
R2: 0.6239428734251422
```

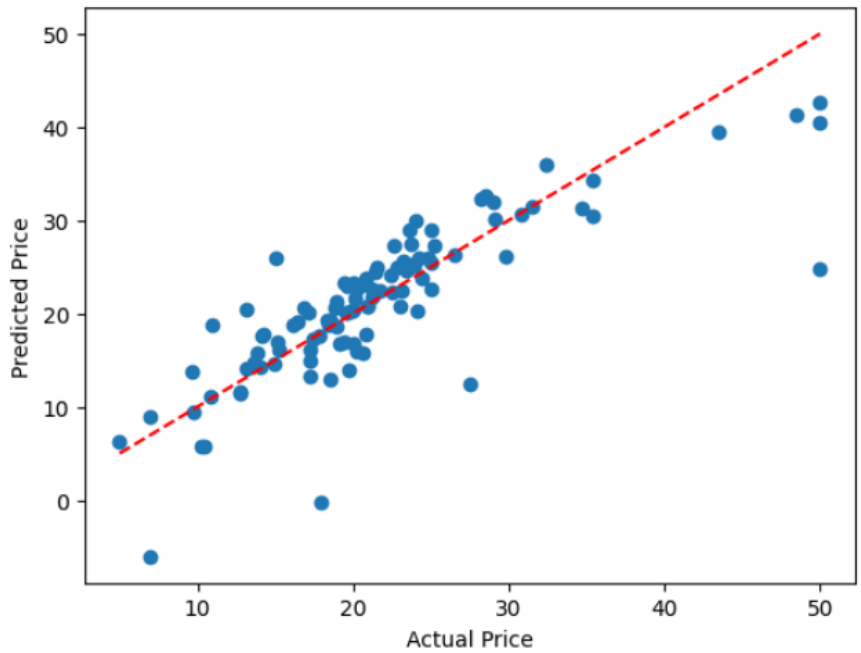



```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.linear_model import Lasso
```

```
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import r2_score

uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0], sep=r'\s+', header=None)

df.columns = ["CRIM","ZN","INDUS","CHAS","NOX","RM","AGE","DIS","RAD","TAX","PTRATIO","B","LSTAT","MEDV"]

target = "MEDV"

X = df.drop(target, axis=1)

y = df[target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)



scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

alphas = np.logspace(-3, 2, 50

grid = GridSearchCV(Lasso(), {'alpha': alphas}, cv=5)

grid.fit(X_train, y_train)

print("Best Alpha:", grid.best_params_)

pred = grid.predict(X_test)

print("R2:", r2_score(y_test, pred))

plt.scatter(y_test, pred)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r')
```

```
plt.xlabel("Actual Price")

plt.ylabel("Predicted Price")

plt.show()
```

```
Choose Files housing.csv
housing.csv(text/csv) - 49082 bytes, last modified: 9/20/2019 - 100% done
Saving housing.csv to housing (17).csv
Best Alpha: {'alpha': np.float64(0.001)}
R2: 0.6687128230636585
```



Conclusion:

In this experiment, we successfully implemented and compared Multi-Linear Regression, Ridge Regression, and Lasso Regression using the Housing dataset. The results showed that features such as number of rooms, location factors, and socio-economic indicators significantly influence house prices. Multi-Linear Regression provided a strong baseline performance, while Ridge and Lasso Regression helped control overfitting through regularization techniques. Hyperparameter tuning further improved model performance and prediction accuracy. Overall, the experiment demonstrated how regression techniques can effectively predict house prices and analyze the impact of multiple influencing factors.